

EASTERN INTERNATIONAL UNIVERSITY



MIS 443

Business Data Management

Assignment 2

PostgreSQL Database Development and SQL Practice

Lecturer: Dang Thai Doan

Prepared by: LLM

Major: Business Analytics

Full Name	IRN	GitHub Repository Link
Đặng Huỳnh Quỳnh Như	2032300287	https://github.com/NhuqDangg/MIS-443---Business-Data-Management/tree/main/MIS443_2032300287_Healthcare/healthcare_folder
Hồ Nhật Anh	2232300196	https://github.com/honhatanhh/MIS-443-

Quarter 4/2025-2026

1. Introduction to the Selected Database Context:

This project recreates the Healthcare schema from SQL Practice Online in PostgreSQL. The database models a simplified clinic environment where patients book appointments with doctors, and doctors record diagnoses for the conditions they treat. The goal is to support common hospital-administration questions: which doctors are busiest, which patients haven't been seen recently, how many appointments happen per department, and so on.

2. Description of the Main Tables and Relationships:

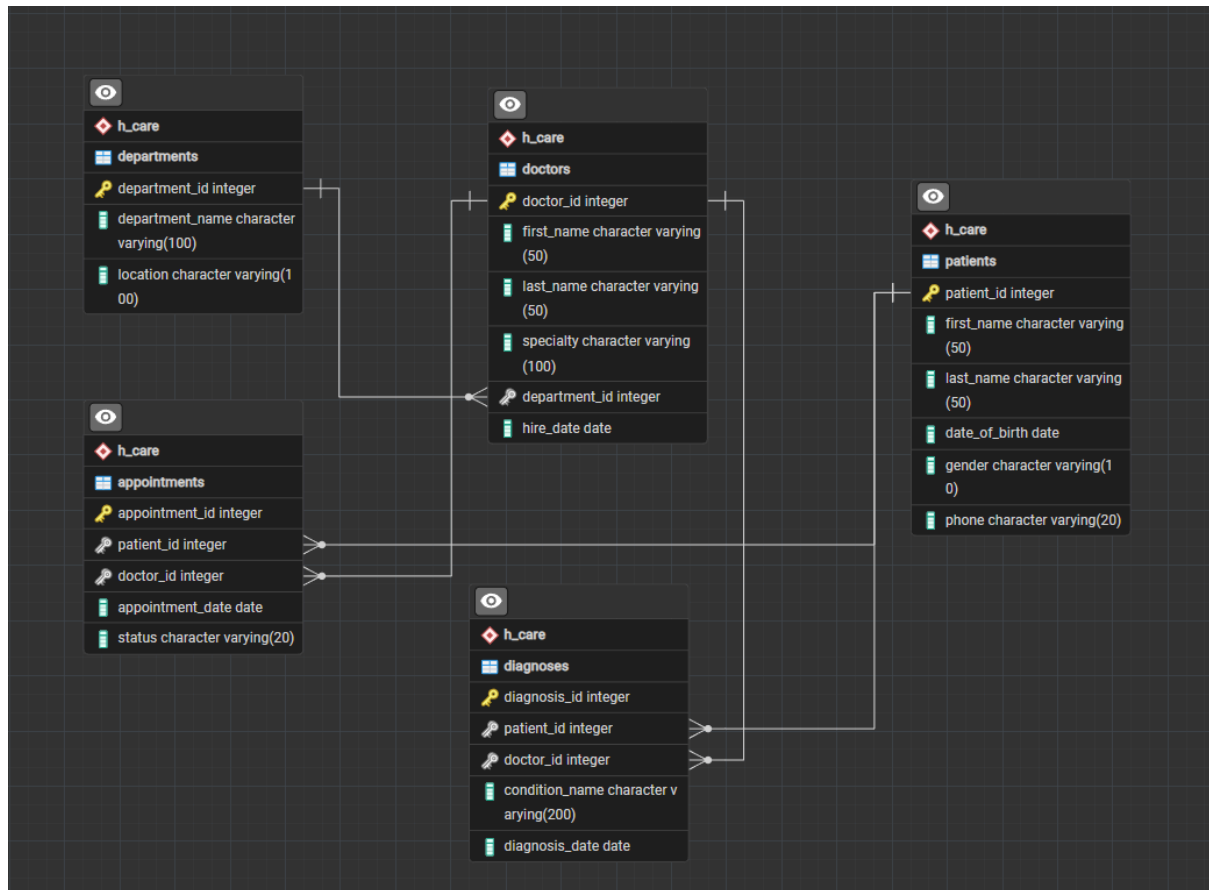
The schema `h_care` contains five tables:

- Departments - department name and physical location.
- Patients -patient profile: name, date of birth, gender, phone.
- Doctors -physician profile: name, specialty, hire date, and the department they belong to.
- Appointments -one row per booked appointment, linking a patient and a doctor, with a date and a status (Scheduled / Completed).
- Diagnoses - one row per diagnosis, linking a patient and a doctor, with the condition name and diagnosis date.

Relationships:

- Departments ->doctors (1:N): one department has many doctors.
- Doctors ->appointments (1:N) and patients -> appointments (1:N): each appointment links exactly one doctor and one patient.
- Doctors -> diagnoses (1:N) and patients -> diagnoses (1:N): each diagnosis links exactly one doctor and one patient.

3. Entity–Relationship Diagram



4. Summary of the Database Implementation:

The database was implemented in three steps, matching the codes/ folder:

- Schema and tables - codes/import_data.sql creates the h_care schema and all five tables, with primary keys on every table and foreign keys enforcing the relationships listed above (doctors.department_id → departments; appointments.patient_id/doctor_id → patients/doctors; diagnoses.patient_id/doctor_id → patients/doctors).
- Data types - INTEGER primary/foreign keys, VARCHAR for names and text fields, DATE for date_of_birth, hire_date, appointment_date and diagnosis_date.
- Data loading - records were imported into each table in an order that satisfies the foreign-key constraints: departments and patients first, then doctors (depends on departments), then appointments and diagnoses (depend on both patients and doctors).

The full script runs successfully end-to-end in pgAdmin 4 and reproduces the table structure required by the 30 SQL Practice Online questions.

5. Summary of the 30 SQL Questions Completed

All 30 questions (codes/exercise.sql) were completed and tested in pgAdmin. Rather than reproducing every query, they are grouped below by the SQL technique they practice:

- Basic retrieval and filtering (Q1–Q4, Q12–Q13): SELECT with WHERE conditions -filtering by department, gender, status, or a date range.
- Sorting (Q5): ORDER BY to rank doctors by hire date.
- Aggregation with GROUP BY / COUNT (Q7, Q9, Q10, Q14, Q18, Q19): counting doctors, appointments, and diagnoses, including per-year and per-patient breakdowns.
- Multi-table JOINS (Q8, Q15–Q17, Q20–Q21): combining patients, doctors, departments, appointments, and diagnoses into single readable rows.
- LEFT JOIN / anti-join patterns for "missing" records (Q22, Q24–Q25): finding patients with no appointments or no diagnoses, using IS NULL after a LEFT JOIN.
- Subqueries and comparison to an average (Q26, Q30): flagging doctors whose appointment count is above the department or hospital-wide average.
- Window functions (Q27–Q28): RANK() OVER (PARTITION BY department) to rank doctors within each department and pull out the top doctor per department.
- CASE WHEN classification (Q29): grouping patients into age bands (Young Adult / Middle-Aged / Senior) and comparing group averages.
- Set operations (Q23): UNION ALL to combine upcoming and past appointments into one labelled list.

6. Selected Screenshots from pgAdmin

1. The registration team is auditing the full patient directory ahead of a system migration
- Return the complete patient roster from the patients table.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postg... x Untitled* x assignment1/postgres@PostgreSQL 18* x

assignment1/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM heal.patients;
```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	patient_id [PK] integer	first_name character varying (50)	last_name character varying (50)	date_of_birth date	gender character varying (10)	phone character varying (20)
1	1001	John	Smith	1985-04-12	M	555-0101
2	1002	Maria	Garcia	1992-08-25	F	555-0102
3	1003	Robert	Johnson	1978-11-03	M	555-0103
4	1004	Jennifer	Lee	1995-01-18	F	555-0104
5	1005	William	Davis	1980-07-30	M	555-0105
6	1006	Amanda	Taylor	1988-12-09	F	555-0106
7	1007	Thomas	Wright	1955-03-10	M	555-0107

Total rows: 7 Query complete 00:00:00.205 CRLF Ln 1, Col 29

- The facilities team needs a directory of all departments and their physical locations for signage updates
 - Return all department names and their locations.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postg... x Untitled* x assignment1/postgres@PostgreSQL 18* x

assignment1/postgres@PostgreSQL 18

Query Query History

```
1 SELECT department_name, location FROM heal.departments;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	department_name character varying (100)	location character varying (100)
1	Cardiology	Building A, Floor 2
2	Emergency	Building A, Floor 1
3	Pediatrics	Building B, Floor 1
4	Surgery	Building A, Floor 3
5	Internal Medicine	Building B, Floor 2

Total rows: 5 Query complete 00:00:00.153 CRLF Ln 1, Col 56

- The cardiology department head needs the current physician roster for the department meeting
 - Return doctors assigned to the Cardiology department.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18*

assignment1/postgres@PostgreSQL 18

Query Query History

```
1 SELECT first_name, last_name, specialty FROM heal.doctors
2 WHERE specialty LIKE '%Cardiologist%';
3
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	specialty character varying (50)
1	Sarah	Chen	Cardiologist
2	David	Kim	Cardiologist

✓ Successfully run. Total query runtime: 186 msec. 2 rows affected. ✕

Total rows: 2 Query complete 00:00:00.186 CRLF Ln 2, Col 39

4. The outreach team is preparing materials for a women's health campaign and needs a contact list of female patients.
-Return all female patients with their contact information.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18*

assignment1/postgres@PostgreSQL 18

Query Query History

```
1 SELECT first_name, last_name, phone FROM heal.patients
2 WHERE gender = 'F';
3
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	phone character varying (20)
1	Maria	Garcia	555-0102
2	Jennifer	Lee	555-0104
3	Amanda	Taylor	555-0106

Total rows: 3 Query complete 00:00:00.126 CRLF Ln 2, Col 20

5. HR needs the physician roster ordered by seniority for a credentialing renewal review.
-Return all doctors sorted by hire date from oldest to most recent.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postg... x Untitled* x assignment1/postgres@PostgreSQL 18* x

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```

1 SELECT first_name, last_name, hire_date FROM heal.doctors
2 ORDER BY hire_date ASC;
3

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	hire_date date
1	Michael	Brown	2017-06-10
2	Emily	Martinez	2018-11-20
3	Sarah	Chen	2019-03-15
4	James	Wilson	2020-07-01
5	David	Kim	2020-09-12
6	Lisa	Anderson	2021-01-05

✓ Successfully run. Total query runtime: 104 msec. 6 rows affected. ✕

Total rows: 6 Query complete 00:00:00.104 CRLF Ln 2, Col 24

6. The quality team is reviewing recently closed patient visits to validate that appointments were properly finalised
- Return all appointments with a Completed status.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postg... x Untitled* x assignment1/postgres@PostgreSQL 18* x

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```

1 SELECT appointment_id, patient_id, appointments.appointment_date FR
2 WHERE status LIKE 'Completed';
3

```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	appointment_id [PK] integer	patient_id integer	appointment_date date
1	1	1001	2025-01-15
2	2	1002	2025-01-16
3	3	1003	2025-01-17
4	8	1002	2025-01-10

✓ Successfully run. Total query runtime: 119 msec. 4 rows affected. ✕

Total rows: 4 Query complete 00:00:00.119 CRLF Ln 2, Col 31

7. The staffing office needs a single headcount of all currently active physicians for a regulatory submission
- Count the total number of doctors on staff.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 18' > 'Databases (2)' > 'assignment1' > 'Schemas (2)' > 'heal'. The 'Query' tab is active, showing the following SQL query:

```
1 SELECT COUNT(doctor_id) AS total_doctors FROM heal.doctors;
2
```

The 'Data Output' tab shows the result of the query:

total_doctors
6

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 108 msec. 1 rows affected.'

8. The epidemiology team is reviewing all new diagnoses made in 2025 for an annual disease incidence report
- Return all diagnoses recorded in the year 2025.

The screenshot shows the pgAdmin 4 interface. The 'Query' tab is active, showing the following SQL query:

```
1 SELECT
2     di.condition_name,
3     di.diagnosis_date,
4     p.last_name
5 FROM heal.diagnoses di
6 JOIN heal.patients p
7     ON di.patient_id = p.patient_id
8 WHERE
9     di.diagnosis_date >= '2025-01-01'
10    AND di.diagnosis_date < '2026-01-01'
11 ORDER BY
12     di.diagnosis_date;
```

The 'Data Output' tab shows the result of the query:

condition_name	diagnosis_date	last_name
Hypertension	2025-01-15	Smith
Seasonal Allergy	2025-01-16	Garcia
Appendicitis	2025-01-17	Johnson

The status bar at the bottom indicates: 'Total rows: 4 Query complete 00:00:00.093'.

9. Count appointments per year using strftime to extract the year from appointment_date. Show year and appointment_count. Order by year.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show the 'heal' database. The 'Query' tab is active, displaying the following SQL query:

```

1 SELECT extract(year FROM appointment_date) AS year,
2 COUNT(appointment_id) AS appointment_count
3 FROM heal.appointments
4 GROUP BY extract(year FROM appointment_date)
5 ORDER BY year ASC;
6

```

The 'Data Output' tab shows the results of the query:

year	appointment_count
2025	9

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.111'.

10. Show the number of distinct diagnoses per patient. Join patients to diagnoses. Show first name, last name, and diagnosis_count. Order by count descending.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show the 'heal' database. The 'Query' tab is active, displaying the following SQL query:

```

1 SELECT p.first_name, p.last_name,
2 COUNT(di.diagnosis_id) AS diagnosis_count
3 FROM heal.patients p
4 JOIN heal.diagnoses di ON p.patient_id = di.patient_id
5 GROUP BY p.patient_id, p.first_name, p.last_name
6 ORDER BY diagnosis_count DESC;
7

```

The 'Data Output' tab shows the results of the query:

first_name	last_name	diagnosis_count
John	Smith	2
Maria	Garcia	1
Robert	Johnson	1
Jennifer	Lee	1
William	Davis	1

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.120'.

11. Find distinct patients who have at least one appointment in 2025. Use strftime to extract the year. Show first name and last name ordered by last name.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* x Untitled* x

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```
1 SELECT DISTINCT p.first_name, p.last_name
2 FROM heal.patients p
3 JOIN heal.appointments ap ON p.patient_id = ap.patient_id
4 WHERE ap.appointment_date >= '2025-01-01' AND ap.appointment_date
5 ORDER BY p.last_name;
6
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)
1	William	Davis
2	Maria	Garcia
3	Robert	Johnson
4	Jennifer	Lee
5	John	Smith
6	Amanda	Taylor

Total rows: 6 Query complete 00:00:00.173 CRLF Ln 5, Col 22

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* x Untitled* x heal.appointments... x

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```
185 ---- Question 11: Find distinct patients who have at least one a
186 SELECT DISTINCT
187     p.first_name,
188     p.last_name
189 FROM
190     heal.patients p
191 JOIN heal.appointments a ON p.patient_id = a.patient_id
192 WHERE
193     TO_CHAR (a.appointment_date, 'YYYY') = '2025'
194 ORDER BY
195     p.last_name;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)
1	William	Davis
2	Maria	Garcia
3	Robert	Johnson
4	Jennifer	Lee

Total rows: 6 Query complete 00:00:00.183 CRLF Ln 195, Col 15

12. The analytics team wants to know what gender values exist in the patient database. List the distinct gender values from the patients table.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* x Untitled* x

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```
1 select distinct(gender) from heal.patients;
2
3
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

gender
M
F

Successfully run. Total query runtime: 483 msec. 2 rows affected. x

Total rows: 2 Query complete 00:00:00.483 CRLF Ln 1, Col 44

13. The scheduling team needs to review all appointments booked during a two-week window. Find all appointments with appointment_date between January 15, 2025 and January 20, 2025 (inclusive). Show appointment_id, patient_id, appointment_date, and status.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* x Untitled* x heal.appointments...

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```
199
200 ---- Question 13: The scheduling team needs to review all appoin
201 SELECT appointment_id, patient_id, appointment_date, status
202 FROM heal.appointments
203 WHERE appointment_date BETWEEN '2025-01-15' AND '2025-01-20';
```

Data Output Messages Notifications

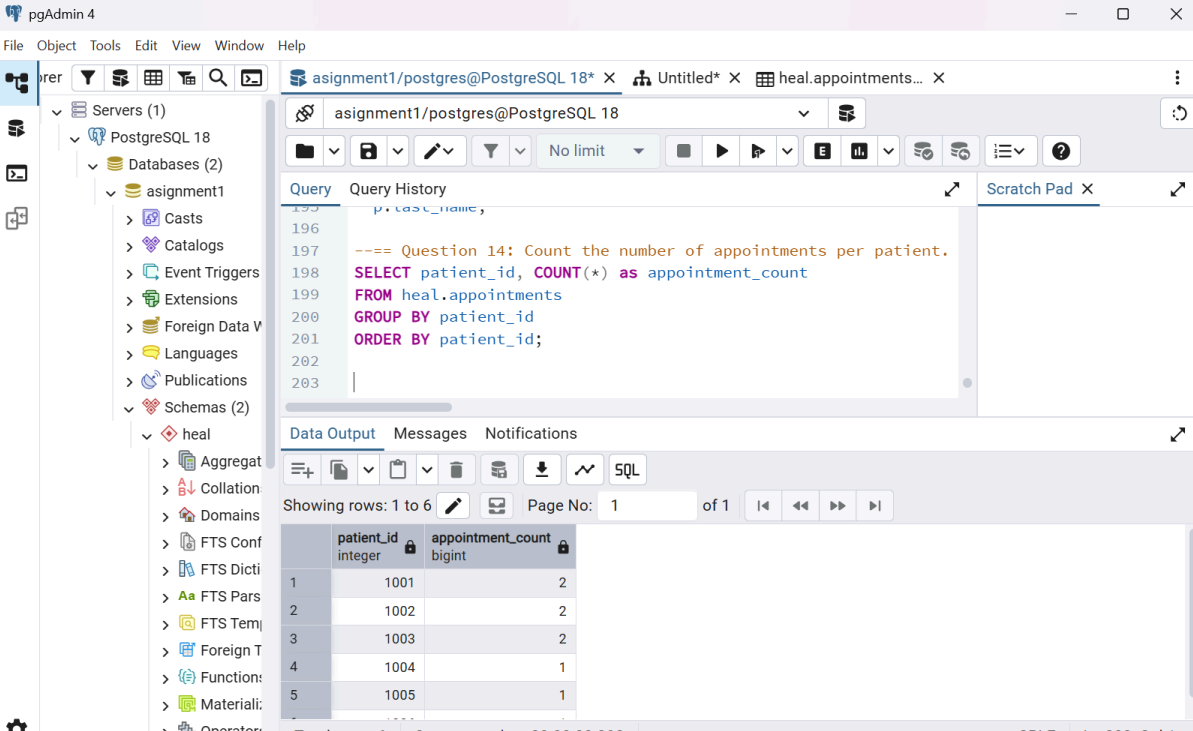
Showing rows: 1 to 6 Page No: 1 of 1

appointment_id [PK] integer	patient_id integer	appointment_date date	status character varying (20)
1	1	2025-01-15	Completed
2	2	2025-01-16	Completed
3	3	2025-01-17	Completed
4	4	2025-01-18	Scheduled
5	5	2025-01-19	Scheduled
6	6	2025-01-20	Scheduled

Total rows: 6 Query complete 00:00:00.264 CRLF Ln 203, Col 62

14. The engagement team is measuring how frequently each patient visits the clinic to flag under-served patients.

Count the number of appointments per patient.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 18', then 'Databases (2)', and finally 'assignment1'. The 'heal' database is selected, and the 'appointments' table is visible in the 'Schemas (2)' list. The main window displays a SQL query in the 'Query' tab:

```
195
196
197 ---- Question 14: Count the number of appointments per patient.
198 SELECT patient_id, COUNT(*) as appointment_count
199 FROM heal.appointments
200 GROUP BY patient_id
201 ORDER BY patient_id;
202
203
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'patient_id' (integer) and 'appointment_count' (bigint). The table shows 5 rows of data:

patient_id	appointment_count
1001	2
1002	2
1003	2
1004	1
1005	1

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.392'.

15. The hospital directory needs each physician listed alongside their department name for the public-facing staff page.

Return each doctor's name paired with their department name.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments... X

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```

211
212 ---- Question 15: The hospital directory needs each physician li
213 --Return each doctor's name paired with their department name.
214 SELECT d.first_name, d.last_name, de.department_name
215 FROM heal.doctors d
216 JOIN heal.departments de ON d.department_id = de.department_id;
217
218

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	department_name character varying (100)
1	Sarah	Chen	Cardiology
2	James	Wilson	Emergency
3	Emily	Martinez	Pediatrics
4	Michael	Brown	Surgery
5	Lisa	Anderson	Internal Medicine
6	David	Kim	Cardiology

Total rows: 6 Query complete 00:00:00.202 CRLF Ln 216, Col 64

16. The clinical records team needs a patient-linked view of all diagnoses for a care history audit.

Return each diagnosis with the patient's first name, last name, condition name, and diagnosis date.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments... X

assignment1/postgres@PostgreSQL 18

Query Query History Scratch Pad X

```

218 ---- Question 16: The clinical records team needs a patient-link
219 --Return each diagnosis with the patient's first name, last name
220 SELECT p.first_name, p.last_name, di.condition_name, di.diagnosi
221 FROM heal.patients p
222 JOIN heal.diagnoses di ON p.patient_id = di.patient_id
223 ORDER BY di.diagnosis_date;
224

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	condition_name character varying (200)	diagnosis_date date
1	John	Smith	High Cholesterol	2024-06-01
2	Jennifer	Lee	Type 2 Diabetes	2024-09-20
3	John	Smith	Hypertension	2025-01-15
4	Maria	Garcia	Seasonal Allergy	2025-01-16
5	Robert	Johnson	Appendicitis	2025-01-17
6	William	Davis	Minor Fracture	2025-01-19

Total rows: 6 Query complete 00:00:00.130 CRLF Ln 223, Col 28

17. The scheduling team needs a forward-looking appointment list with both patient and doctor names for daily briefing sheets.
Return all scheduled appointments with patient and doctor full names.

The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 database named 'assignment1'. A query is executed in the 'Query' tab, returning a list of scheduled appointments with patient and doctor names.

```

226 --Return all scheduled appointments with patient and doctor full
227 SELECT a.appointment_date,
228        p.first_name || ' ' || p.last_name AS patient_name,
229        d.first_name || ' ' || d.last_name AS doctor_name
230 FROM heal.appointments a JOIN heal.patients p ON a.patient_id =
231      JOIN heal.doctors d ON a.doctor_id = d.doctor_id
232 WHERE a.status = 'Scheduled' ORDER BY a.appointment_date;

```

The 'Data Output' tab shows the following results:

	appointment_date	patient_name	doctor_name
1	2025-01-18	Jennifer Lee	Sarah Chen
2	2025-01-19	William Davis	James Wilson
3	2025-01-20	Amanda Taylor	Lisa Anderson
4	2025-02-01	John Smith	David Kim

Successfully run. Total query runtime: 1 secs 196 msec. 4 rows affected.

Total rows: 4 Query complete 00:00:01.196 CRLF Ln 232, Col 58

18. The capacity planning team needs to know physician headcount per department to identify under-staffed units.
Return the number of doctors in each department.

The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 database named 'assignment1'. A query is executed in the 'Query' tab, returning the number of doctors in each department.

```

233
234 ---- Question 18: The capacity planning team needs to know physi
235 --Return the number of doctors in each department.
236 SELECT de.department_name, COUNT(d.doctor_id) AS doctor_count FR
237 LEFT JOIN heal.doctors d ON de.department_id = d.department_id
238 GROUP BY de.department_name
239 ORDER BY department_name;

```

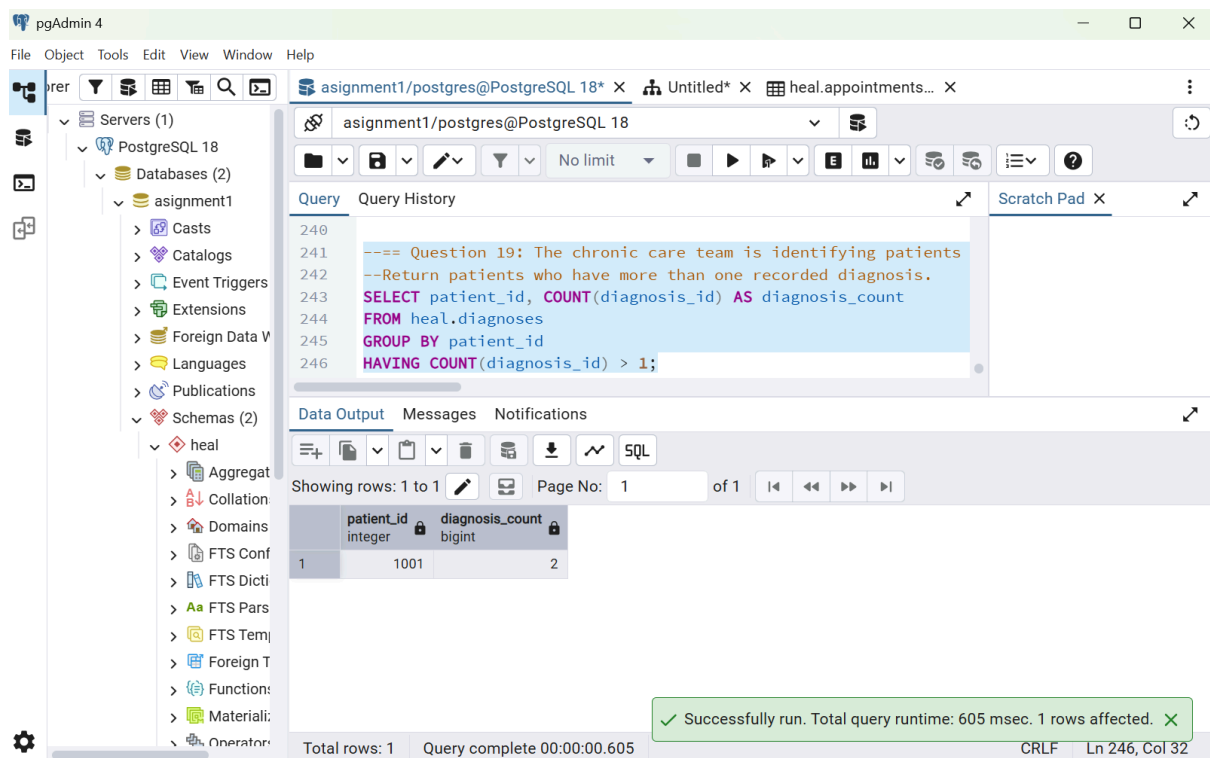
The 'Data Output' tab shows the following results:

	department_name	doctor_count
1	Cardiology	2
2	Emergency	1
3	Internal Medicine	1
4	Pediatrics	1
5	Surgery	1

Total rows: 5 Query complete 00:00:00.162 CRLF Ln 234, Col 17

19. The chronic care team is identifying patients who carry multiple diagnoses to prioritise case management outreach.

Return patients who have more than one recorded diagnosis.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' > 'Databases (2)' > 'assignment1'. The 'Query' tab is active, displaying the following SQL query:

```
--== Question 19: The chronic care team is identifying patients
--Return patients who have more than one recorded diagnosis.
SELECT patient_id, COUNT(diagnosis_id) AS diagnosis_count
FROM heal.diagnoses
GROUP BY patient_id
HAVING COUNT(diagnosis_id) > 1;
```

The 'Data Output' tab shows the results of the query in a table with two columns: 'patient_id' (integer) and 'diagnosis_count' (bigint). The table contains one row:

patient_id	diagnosis_count
1	2

At the bottom, a status bar indicates 'Total rows: 1', 'Query complete 00:00:00.605', and a green message box stating 'Successfully run. Total query runtime: 605 msec. 1 rows affected.'

20. The patient flow team needs each appointment linked to the department responsible so throughput can be measured by clinical unit.

Return each appointment's date, patient last name, and the department of the attending doctor.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments...

assignment1/postgres@PostgreSQL 18

Query Query History

```

249 --Return each appointment's date, patient last name, and the dep
250 SELECT a.appointment_date, p.last_name AS patient_last_name, de.
251 FROM heal.appointments a
252 JOIN heal.patients p ON a.patient_id = p.patient_id
253 JOIN heal.doctors d ON a.doctor_id = d.doctor_id
254 JOIN heal.departments de ON d.department_id = de.department_id
255 ORDER BY a.appointment_date, p.last_name;

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

	appointment_date date	patient_last_name character varying (50)	department_name character varying (100)
1	2025-01-10	Garcia	Pediatrics
2	2025-01-12	Johnson	Internal Medicine
3	2025-01-15	Smith	Cardiology
4	2025-01-16	Garcia	Pediatrics
5	2025-01-17	Johnson	Surgery
6	2025-01-18	Lee	Cardiology

Total rows: 9 Query complete 00:00:00.292 CRLF Ln 255, Col 42

21. The hospital audit team needs a complete view of every appointment. Show full appointment details including patient name, doctor name, department name, appointment date, and status. Order by appointment date.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments...

assignment1/postgres@PostgreSQL 18

Query Query History

```

256
257 ---- Question 21: The hospital audit team needs a complete view
258 SELECT p.first_name, p.last_name, d.first_name AS dr_first, d.la
259 de.department_name, a.appointment_date, a.status
260 FROM heal.appointments a
261 JOIN heal.patients p ON a.patient_id = p.patient_id
262 JOIN heal.doctors d ON a.doctor_id = d.doctor_id
263 JOIN heal.departments de ON d.department_id = de.department_id
264 ORDER BY a.appointment_date;

```

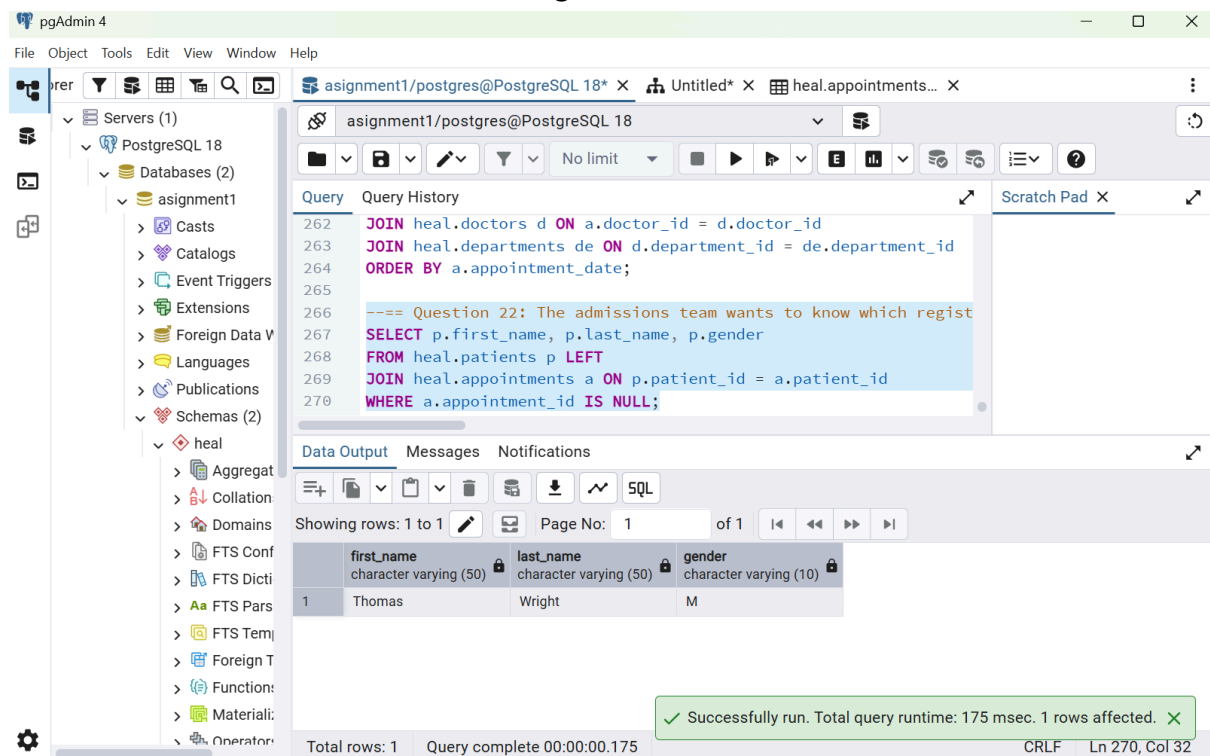
Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	dr_first character varying (50)	dr_last character varying (50)	department_name character varying (100)	a
1	Maria	Garcia	Emily	Martinez	Pediatrics	2
2	Robert	Johnson	Lisa	Anderson	Internal Medicine	2
3	John	Smith	Sarah	Chen	Cardiology	2
4	Maria	Garcia	Emily	Martinez	Pediatrics	2

Total rows: 9 Query complete 00:00:00.189 CRLF Ln 264, Col 29

22. The admissions team wants to know which registered patients have never had an appointment scheduled. Find patients who have no appointment records. Show first name, last name, and gender.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' > 'Databases (2)' > 'assignment1'. The 'heal' schema is selected. The main pane shows a SQL query editor with the following query:

```

262 JOIN heal.doctors d ON a.doctor_id = d.doctor_id
263 JOIN heal.departments de ON d.department_id = de.department_id
264 ORDER BY a.appointment_date;
265
266 ---- Question 22: The admissions team wants to know which regist
267 SELECT p.first_name, p.last_name, p.gender
268 FROM heal.patients p LEFT
269 JOIN heal.appointments a ON p.patient_id = a.patient_id
270 WHERE a.appointment_id IS NULL;

```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with the following columns: first_name, last_name, and gender. The table contains one row of data:

first_name	last_name	gender
Thomas	Wright	M

At the bottom of the interface, a status bar indicates: 'Total rows: 1', 'Query complete 00:00:00.175', and a green message box says 'Successfully run. Total query runtime: 175 msec. 1 rows affected.'

23. Create a unified appointment list by combining Scheduled (labelled 'Upcoming') and Completed (labelled 'Past') appointments. Show patient_id, doctor_id, appointment_date, and category. Order by appointment_date.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments... X

assignment1/postgres@PostgreSQL 18

Query Query History

```

270 WHERE a.appointment_id IS NULL;
271
272 ---- Question 23: Create a unified appointment list by combining
273 SELECT patient_id, doctor_id, appointment_date,
274 'Upcoming' AS category FROM heal.appointments WHERE status = 'Sc
275 UNION ALL SELECT patient_id, doctor_id, appointment_date,
276 'Past' AS category FROM heal.appointments WHERE status = 'Comple
277 ORDER BY appointment_date;

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	patient_id integer	doctor_id integer	appointment_date date	category text
1	1002	103	2025-01-10	Past
2	1001	101	2025-01-15	Past
3	1002	103	2025-01-16	Past
4	1003	104	2025-01-17	Past
5	1004	101	2025-01-18	Upcoming
6	1005	102	2025-01-19	Upcoming

Total rows: 8 Query complete 00:00:00.167 CRLF Ln 277, Col 27

24. The clinical team wants to identify patients who have never received a diagnosis. Find all patients who have no records in the diagnoses table. Show first name, last name, and gender.

pgAdmin 4

File Object Tools Edit View Window Help

assignment1/postgres@PostgreSQL 18* X Untitled* X heal.appointments... X

assignment1/postgres@PostgreSQL 18

Query Query History

```

277 ORDER BY appointment_date;
278
279 ---- Question 24: The clinical team wants to identify patients w
280 SELECT p.first_name, p.last_name, p.gender
281 FROM heal.patients p
282 LEFT JOIN heal.diagnoses di ON p.patient_id = di.patient_id
283 WHERE di.diagnosis_id IS NULL;

```

Data Output Messages Notifications

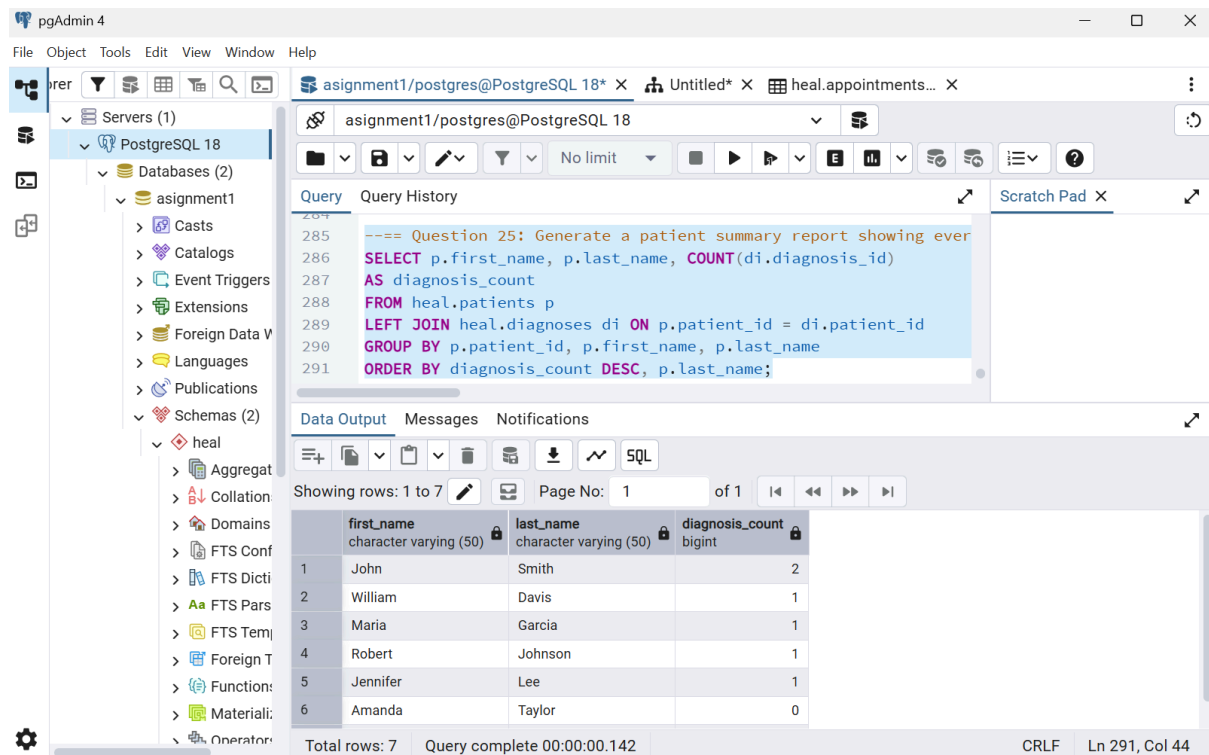
Showing rows: 1 to 2 Page No: 1 of 1

	first_name character varying (50)	last_name character varying (50)	gender character varying (10)
1	Thomas	Wright	M
2	Amanda	Taylor	F

Successfully run. Total query runtime: 395 msec. 2 rows affected. X

Total rows: 2 Query complete 00:00:00.395 CRLF Ln 283, Col 31

25. Generate a patient summary report showing every registered patient alongside how many diagnoses they have received, including patients with zero diagnoses. Show first name, last name, and diagnosis_count. Order by diagnosis_count descending, then last name.



The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is as follows:

```

285 ---- Question 25: Generate a patient summary report showing ever
286 SELECT p.first_name, p.last_name, COUNT(di.diagnosis_id)
287 AS diagnosis_count
288 FROM heal.patients p
289 LEFT JOIN heal.diagnoses di ON p.patient_id = di.patient_id
290 GROUP BY p.patient_id, p.first_name, p.last_name
291 ORDER BY diagnosis_count DESC, p.last_name;

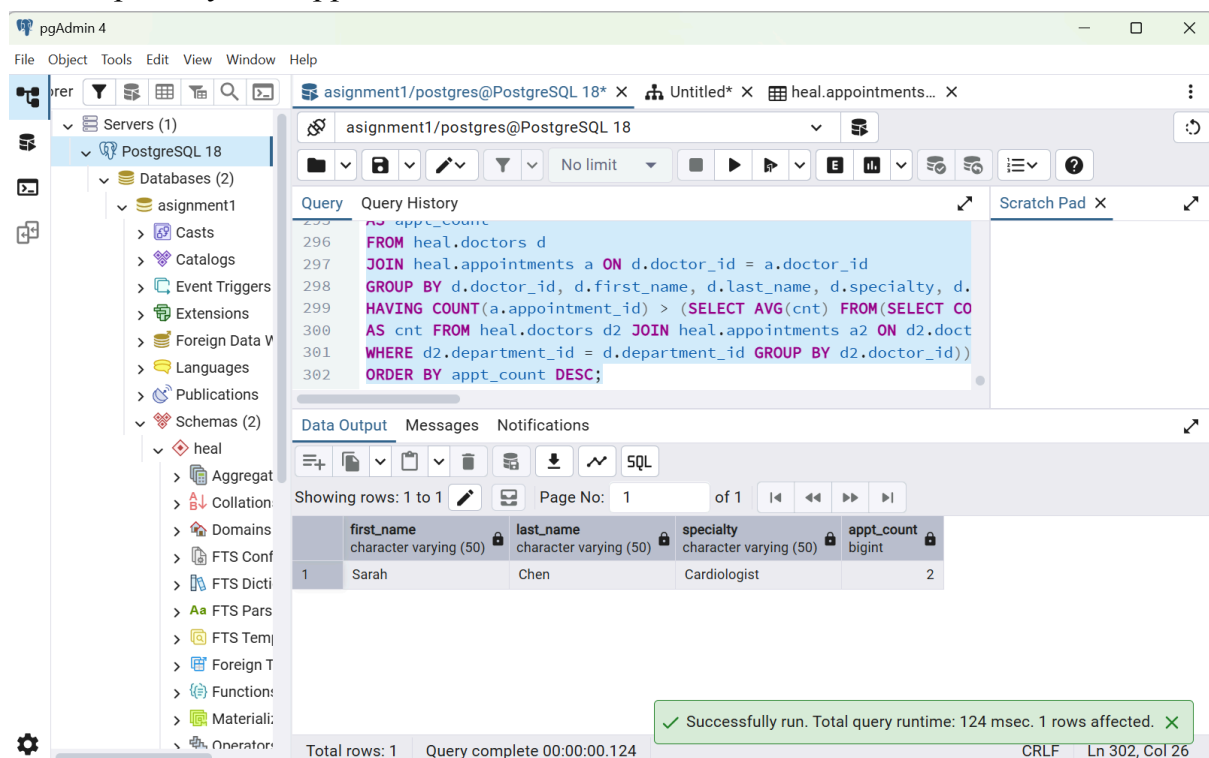
```

The Data Output tab shows the following results:

	first_name	last_name	diagnosis_count
1	John	Smith	2
2	William	Davis	1
3	Maria	Garcia	1
4	Robert	Johnson	1
5	Jennifer	Lee	1
6	Amanda	Taylor	0

Total rows: 7. Query complete 00:00:00.142. CRLF Ln 291, Col 44

26. Find doctors who have more appointments than the average number of appointments for all doctors in their department. Show first name, last name, specialty, and appointment count.



The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is as follows:

```

296 FROM heal.doctors d
297 JOIN heal.appointments a ON d.doctor_id = a.doctor_id
298 GROUP BY d.doctor_id, d.first_name, d.last_name, d.specialty, d.
299 HAVING COUNT(a.appointment_id) > (SELECT AVG(cnt) FROM (SELECT CO
300 AS cnt FROM heal.doctors d2 JOIN heal.appointments a2 ON d2.doct
301 WHERE d2.department_id = d.department_id GROUP BY d2.doctor_id))
302 ORDER BY appt_count DESC;

```

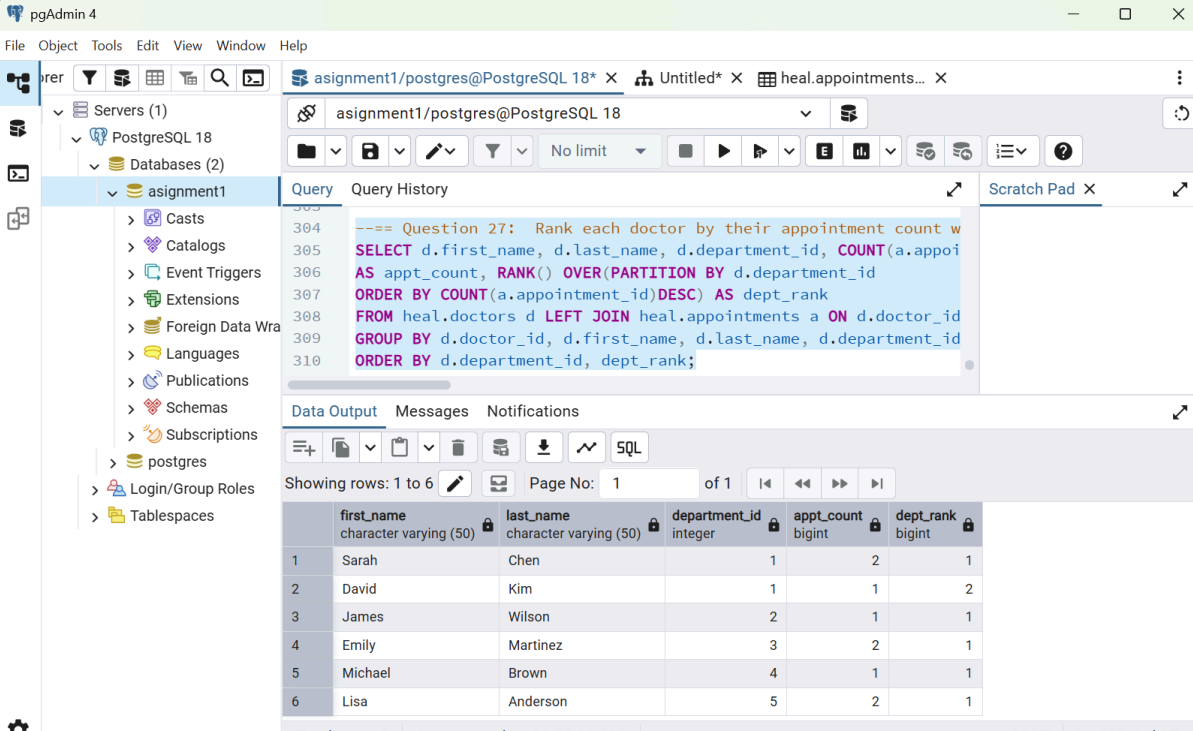
The Data Output tab shows the following results:

	first_name	last_name	specialty	appt_count
1	Sarah	Chen	Cardiologist	2

Total rows: 1. Query complete 00:00:00.124. CRLF Ln 302, Col 26

Successfully run. Total query runtime: 124 msec. 1 rows affected.

27. Rank each doctor by their appointment count within their department. Show first name, last name, department_id, appt_count, and dept_rank. Order by department, then rank.



The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is as follows:

```

---- Question 27: Rank each doctor by their appointment count w
SELECT d.first_name, d.last_name, d.department_id, COUNT(a.appoi
AS appt_count, RANK() OVER(PARTITION BY d.department_id
ORDER BY COUNT(a.appointment_id)DESC) AS dept_rank
FROM heal.doctors d LEFT JOIN heal.appointments a ON d.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name, d.department_id
ORDER BY d.department_id, dept_rank;

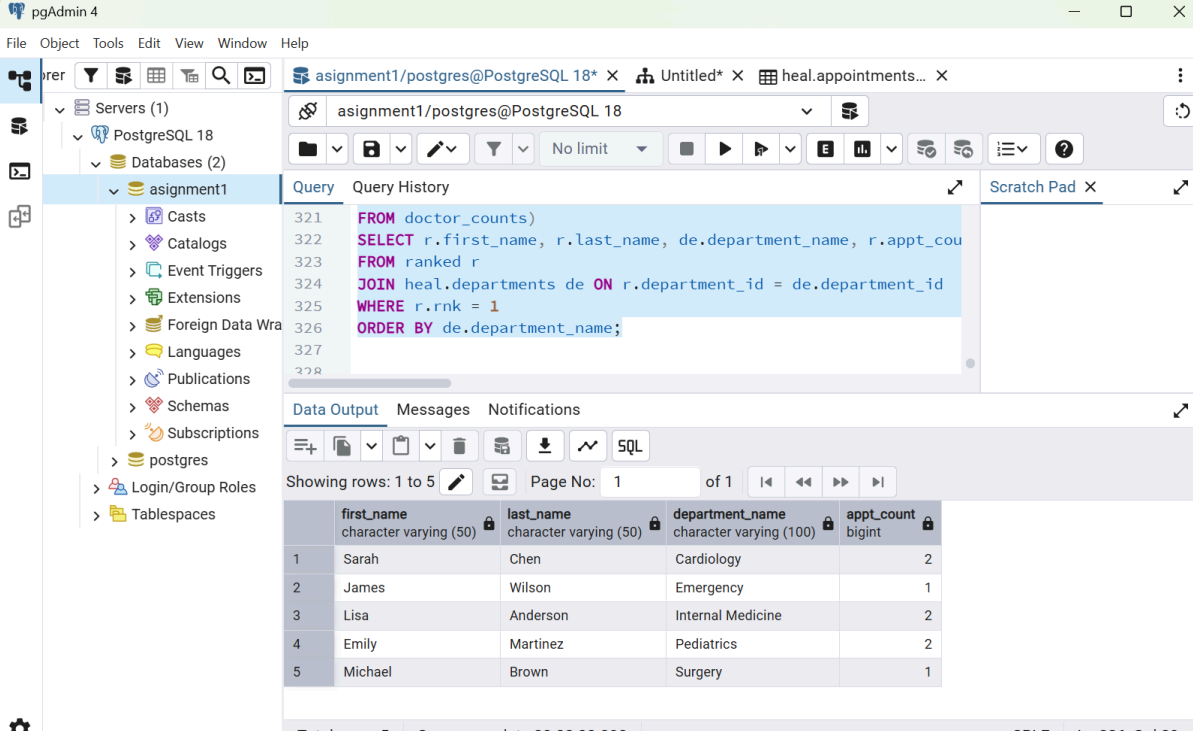
```

The Data Output tab shows the following results:

	first_name	last_name	department_id	appt_count	dept_rank
1	Sarah	Chen	1	2	1
2	David	Kim	1	1	2
3	James	Wilson	2	1	1
4	Emily	Martinez	3	2	1
5	Michael	Brown	4	1	1
6	Lisa	Anderson	5	2	1

Total rows: 6 Query complete 00:00:00.121 CRLF Ln 310, Col 37

28. Find the doctor with the most appointments in each department. Show doctor name, department name, and appointment count.



The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is as follows:

```

FROM doctor_counts)
SELECT r.first_name, r.last_name, de.department_name, r.appt_cou
FROM ranked r
JOIN heal.departments de ON r.department_id = de.department_id
WHERE r.rnk = 1
ORDER BY de.department_name;

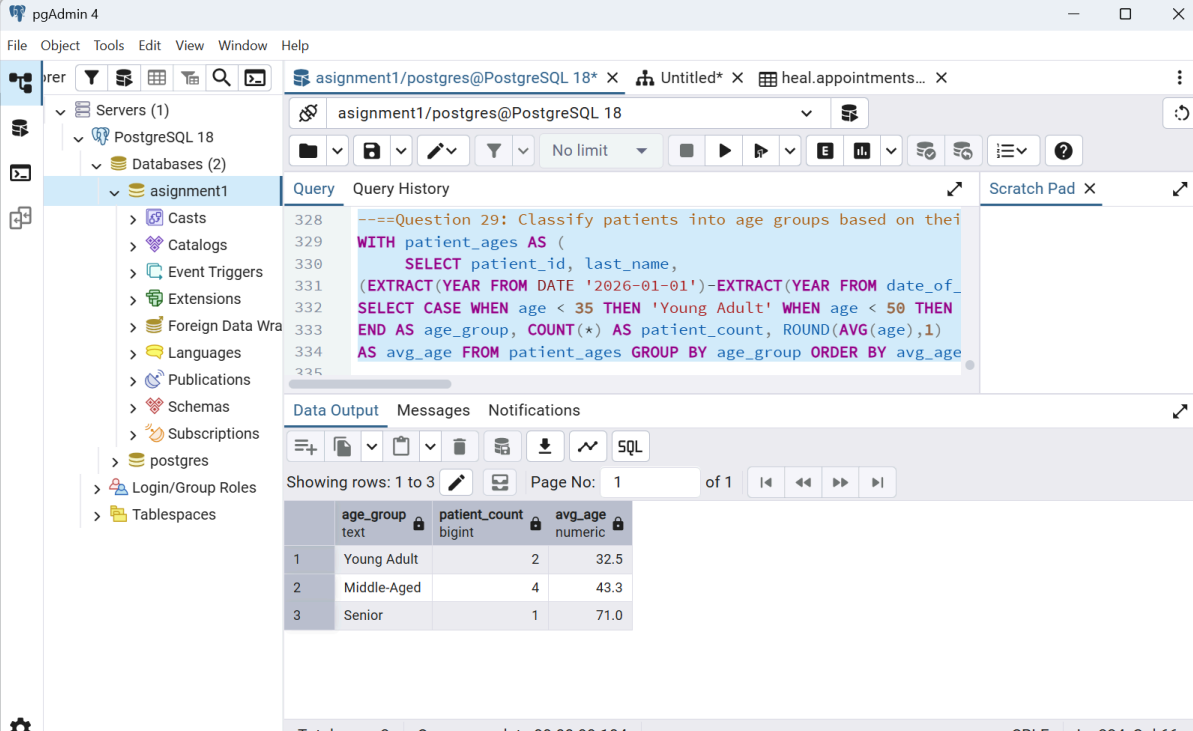
```

The Data Output tab shows the following results:

	first_name	last_name	department_name	appt_count
1	Sarah	Chen	Cardiology	2
2	James	Wilson	Emergency	1
3	Lisa	Anderson	Internal Medicine	2
4	Emily	Martinez	Pediatrics	2
5	Michael	Brown	Surgery	1

Total rows: 5 Query complete 00:00:00.090 CRLF Ln 326, Col 29

29. Classify patients into age groups based on their age as of 2026-01-01: Young Adult (under 35), Middle-Aged (35–49), Senior (50+). Show each group's patient count and average age. Order by avg_age ascending.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' and 'assignment1'. The 'assignment1' database is selected. The 'Query' tab is active, displaying the following SQL query:

```

328 ----Question 29: Classify patients into age groups based on thei
329 WITH patient_ages AS (
330     SELECT patient_id, last_name,
331     (EXTRACT(YEAR FROM DATE '2026-01-01')-EXTRACT(YEAR FROM date_of_
332     SELECT CASE WHEN age < 35 THEN 'Young Adult' WHEN age < 50 THEN
333     END AS age_group, COUNT(*) AS patient_count, ROUND(AVG(age),1)
334     AS avg_age FROM patient_ages GROUP BY age_group ORDER BY avg_age
335
  
```

The 'Data Output' tab shows the results of the query. The results are displayed in a table with the following columns: age_group, patient_count, and avg_age. The results are ordered by avg_age ascending.

age_group	patient_count	avg_age
1 Young Adult	2	32.5
2 Middle-Aged	4	43.3
3 Senior	1	71.0

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.104'.

30. The operations team wants to flag doctors who are busier than the hospital average. Find doctors whose appointment count exceeds the hospital-wide average appointment count per doctor. Show first name, last name, specialty, and appointment count.

The screenshot shows the pgAdmin 4 web interface. On the left, a tree view shows the database structure: Servers (1) > PostgreSQL 18 > Databases (2) > assignment1. The main pane displays a SQL query in the 'Query' tab. The query is a window function query that ranks doctors by the number of appointments they have, grouped by department. The query is as follows:

```

339 FROM heal.doctors d JOIN heal.appointments a
340 ON d.doctor_id = a.doctor_id
341 GROUP BY d.doctor_id, d.first_name, d.last_name, d.specialty
342 HAVING COUNT(a.appointment_id) > (SELECT AVG(cnt)
343 FROM (SELECT COUNT(appointment_id) AS cnt
344 FROM heal.appointments GROUP BY doctor_id))
345 ORDER BY appt_count DESC;
346

```

Below the query editor, the 'Data Output' tab shows the results of the query. It displays 3 rows of data. The columns are: first_name, last_name, specialty, and appt_count. The results are as follows:

	first_name character varying (50)	last_name character varying (50)	specialty character varying (50)	appt_count bigint
1	Sarah	Chen	Cardiologist	2
2	Emily	Martinez	Pediatrician	2
3	Lisa	Anderson	Internal Medicine	2

At the bottom of the interface, a status bar shows 'Total rows: 3' and 'Query complete 00:00:00.118'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 118 msec. 3 rows affected.'

7. Challenges Encountered and Solutions Applied:

Unfamiliar advanced SQL commands: Window functions and grouping commands, such as RANK(), HAVING, and PARTITION BY, were new to us. It took a significant amount of time to understand how they worked and to write them correctly, especially for questions that required ranking doctors within each department.

Confusion between JOIN types: We frequently mixed up the different types of JOIN (INNER JOIN, LEFT JOIN), which caused errors when running queries -particularly for questions that required finding patients with no appointments or no diagnoses, where a LEFT JOIN was needed instead of a regular JOIN.

To resolve these issues, we reviewed the PostgreSQL documentation and worked through smaller, simpler examples of each command before applying them to the actual questions. We also tested each query step by step in pgAdmin and compared the results against the expected output on SQL Practice Online to confirm correctness before finalising the SQL file.

8. Responsibilities and Contributions of Each Member:

Member	Contribution
Đặng Huỳnh Quỳnh Như	Wrote and tested the initial queries on SQL Practice Online, prepared the Word report, organised and published the project on GitHub, and completed Questions 11–20 in pgAdmin 4.
Hồ Nhật Anh	Completed the main SQL coding in pgAdmin 4 for Questions 1–10 and 21–30.

9. Conclusion:

This project reinforced the full lifecycle of a small relational database: interpreting a schema, enforcing it with keys and constraints, loading data in a dependency-safe order, and writing SQL that answers real operational questions - from simple filters to window-function rankings. The Healthcare schema in particular required combining multiple join patterns (direct joins, anti-joins for missing records, and self-referencing comparisons to a group average) to answer questions a hospital administrator would realistically ask.

10. GitHub Repository:

https://github.com/NhuqDang/MIS-443---Business-Data-Management/tree/main/MIS443_2032300287_Healthcare